DroneArjuna GCS

Vehicle Integration Guide

MAVLink connection to the hosted DroneArjuna platform

Document	Vehicle Integration Guide (interface specification for firmware / integration engineers)
Version / date	1.0 / 2026-10-07
Audience	Firmware, autopilot-integration and companion-computer engineers of the vehicle vendor
Scope	How a vehicle connects over the internet (or any IP link) to the hosted DroneArjuna server using MAVLink 2
Distribution	Controlled. Share only with the vendor assigned to the vehicle. Contains no secrets - the signing key is delivered separately.

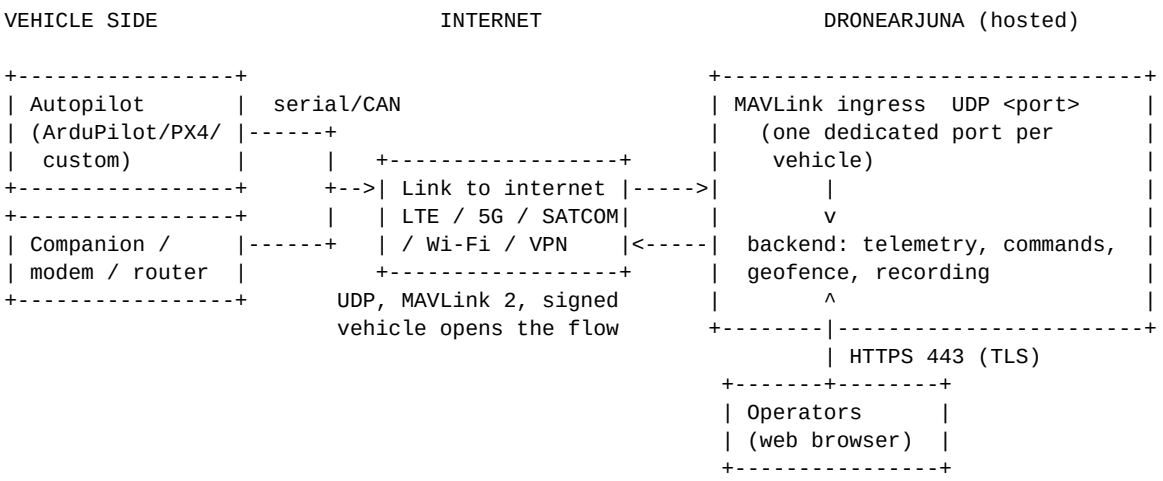
BEFORE YOU START: Items marked **<<DRONEARJUNA TO FILL>>** are filled in by the DroneArjuna team in Section 3 before this document is sent. Do not start integration until Section 3 is complete.

1. Purpose and scope

DroneArjuna is a web-based ground control station. Its server is hosted centrally and operators use it through a browser. Vehicles do **not** connect to an operator's PC: each vehicle streams MAVLink over the network to the DroneArjuna server, and the server sends operator commands back over the same flow.

This guide defines everything the vehicle side must do for that to work: network path, MAVLink profile, required telemetry, accepted commands, timing, security, and the acceptance tests we run together before first flight. DroneArjuna requires no DroneArjuna-specific code in the vehicle - it is a standard MAVLink ground station.

2. Architecture



- **The vehicle initiates the connection.** It sends MAVLink to the server's public address and port. The server answers to the source address and port it sees, so this works from behind NAT, carrier-grade NAT and mobile networks without any inbound port on the vehicle side.
- **One vehicle = one UDP port.** The server listens on a separate UDP port per vehicle (assigned in Section 3). Two vehicles must never send to the same port.
- **One MAVLink system on that port.** Only the vehicle's autopilot may be visible on the flow (see Section 5.3).
- **Transport is UDP.** TCP, serial and HF-modem transports exist in the platform but are not the standard hosted path; contact us if your link needs one of them.

3. Connection parameters (filled in by DroneArjuna)

Parameter	Value
Server hostname / IP	**<<DRONEARJUNA TO FILL>>**
Destination UDP port (this vehicle)	**<<DRONEARJUNA TO FILL>>**
Protocol	MAVLink 2 over UDP, vehicle to server (outbound from the vehicle)
Vehicle MAVLink system id (SYSID)	**<<DRONEARJUNA TO FILL>>**
Signing key	Delivered separately through an agreed secure channel. Never sent by e-mail together with this document. **<<DRONEARJUNA TO FILL>>**
Allow-listed vehicle egress IP (if used)	**<<DRONEARJUNA TO FILL>>**
Staging server for testing	**<<DRONEARJUNA TO FILL>>**
DroneArjuna integration contact	**<<DRONEARJUNA TO FILL>>**
Test window	**<<DRONEARJUNA TO FILL>>**

Information we need from you

Item	Why
Autopilot and firmware version (ArduPilot Copter/Plane, PX4, or custom)	Flight-mode names are decoded per autopilot type
Vehicle type and unique SYSID	Registered in the platform; must match what your autopilot sends
Link type (LTE/5G/SATCOM/Wi-Fi/VPN) and expected bandwidth and latency	Sizing and timeout review
Public egress IP or range of the vehicle link	Only if we apply source allow-listing
Companion-computer software used to forward MAVLink	To agree filtering and forwarding configuration
A recent flight log (.tlog / .bin) with the intended message set	Lets us review your stream before the first live test

4. Network requirements

Requirement	Detail
Direction	Outbound UDP from vehicle to :. No inbound rule needed on the vehicle side.
Keep-alive	Send traffic continuously (the HEARTBEAT at 1 Hz does this). Mobile NAT mappings expire after a short idle time; if the flow goes quiet, server-to-vehicle commands cannot reach the vehicle.
Persistence	Do not send a single packet and wait. The server opens the listener when the vehicle is connected from the operator side (or by its automatic connector, which retries about every 15 s). Keep sending HEARTBEAT and telemetry; ignore 'port unreachable' replies and keep retrying.
Bandwidth (estimate)	About 1.3 KB/s (about 11 kbit/s) vehicle to server for the stream set in Section 6 with signing on, plus about 0.03 KB/s for the server heartbeat. Add whatever else your firmware streams and plan 2-3x headroom.
Silence tolerance	The server drops the link after 10 s with no message at all (Section 8). Design the link so gaps stay well under that.
Packet size	Standard MAVLink 2 frames, under 300 bytes, well under any MTU. Do not fragment or batch.
Time	No time sync with the server is needed beyond a reasonable clock for signing timestamps (Section 9).

SECURITY: MAVLink over UDP is **not encrypted**. Signing authenticates and protects integrity, but telemetry can be read in transit. If confidentiality is required, run the link through a VPN (for example WireGuard) between the vehicle router and the server network and tell us - we will give you the tunnel endpoint instead of a public address.

5. MAVLink profile

5.1 Protocol

- **MAVLink 2 only.** The server always runs in MAVLink 2 mode; this is required for signing. MAVLink 1 frames are not supported.
- Standard common dialect. No custom messages are required.

5.2 Identities

Role	System id	Component / type	Notes
DroneArjuna server (GCS)	255	MAV_TYPE_GCS, MAV_AUTOPILOT_INVALID	Sends HEARTBEAT at 1 Hz while connected. ArduPilot's default SYSID_MYGCS is also 255 - keep it.
Vehicle	Unique per vehicle (Section 3)	Autopilot component, normally 1 (MAV_COMP_ID_AUTOPILOT1)	The server addresses commands to the system and component of the first HEARTBEAT it receives.

5.3 One autopilot per link - important

The server treats **every** HEARTBEAT on the flow as the vehicle's own. A HEARTBEAT from a gimbal, camera, companion computer or other component will overwrite the displayed flight mode and armed state. Forward only the autopilot's messages (system id of the vehicle, autopilot component) to DroneArjuna, or filter other components' heartbeats at the companion computer. The self-check tool in Appendix B reports this problem.

Likewise, if two vehicles share one flow they will be mixed into one record. One vehicle per port, always.

6. Vehicle to server: required telemetry

At connection the server requests these rates using MAV_CMD_SET_MESSAGE_INTERVAL (and the legacy REQUEST_DATA_STREAM). **Do not depend on the request:** configure your firmware so that these are already streaming at these rates after boot. The request is sent once at connection; if the vehicle reboots without the link dropping, the server does not repeat it.

Message	Rate	Required	Fields DroneArjuna uses and units
HEARTBEAT	1 Hz	Yes	type, autopilot, base_mode (armed bit), custom_mode (flight mode), system_status. Autopilot must be ARDUPILOTMEGA or PX4 for mode names; otherwise modes show as MODE_<n>.
GLOBAL_POSITION_INT	4 Hz	Yes	lat/lon (degE7), alt (mm MSL), relative_alt (mm), hdg (cdeg), vx/vy/vz (cm/s)
ATTITUDE	10 Hz	Yes	roll, pitch, yaw (rad) and rates
VFR_HUD	4 Hz	Yes	airspeed, groundspeed, altitude, climb, throttle
SYS_STATUS	2 Hz	Yes	voltage_battery (mV), current_battery (cA), battery_remaining (%), load (d%)
GPS_RAW_INT	2 Hz	Yes	fix_type, satellites_visible, eph (cm; HDOP x 100)
RC_CHANNELS	2 Hz	Optional	rssi (0-254; 255 = unknown), shown as link signal
BATTERY_STATUS	about 0.2-1 Hz	Optional	battery_remaining (%; -1 = unknown)
COMMAND_ACK	On every command	Yes	command, result. See Section 7.
RADIO_STATUS	As produced	Optional	Accepted unsigned even when signing is on

Data quality rules that affect behaviour:

- The server refuses to ARM unless the vehicle reports a 3D GPS fix (fix_type 3 or better) with at least 6 satellites and battery at or above 20%. If battery_remaining is reported as -1 (unknown), arming is judged on the value the server holds - report a valid percentage.
- Geofence breach detection runs on GLOBAL_POSITION_INT. Missing or stale position means no geofence protection.
- Report SI-correct units exactly as the MAVLink definition states; the server applies fixed scale factors.

7. Server to vehicle: commands your firmware must accept

All commands are addressed to the vehicle's system id and the component seen in its first HEARTBEAT, and are sent from system id 255.

Operator action	MAVLink sent	Parameters	Expected vehicle response
Arm	COMMAND_LONG MAV_CMD_COMPONENT_ARM_DISARM	param1 = 1	COMMAND_ACK within 5 s
Disarm	COMMAND_LONG MAV_CMD_COMPONENT_ARM_DISARM	param1 = 0	COMMAND_ACK within 5 s

Operator action	MAVLink sent	Parameters	Expected vehicle response
Emergency stop	COMMAND_LONG MAV_CMD_DO_FLIGHTTERMINATION	param1 = 1	COMMAND_ACK; motors cut
Takeoff	COMMAND_LONG MAV_CMD_NAV_TAKEOFF	param7 = altitude (m, default 30); allowed 1-500 m; vehicle must be armed	COMMAND_ACK
Set mode	SET_MODE with MAV_MODE_FLAG_CUSTOM_MODE_ENABLED	custom_mode = number from the autopilot's mode table	Mode change visible in the next HEARTBEAT (no ACK needed)
Return to launch	SET_MODE to RTL (or SMART_RTL)	-	HEARTBEAT shows the new mode
Land	SET_MODE to LAND	-	HEARTBEAT shows the new mode
Go to	MISSION_ITEM_INT, frame GLOBAL_RELATIVE_ALT_INT, command NAV_WAYPOINT, current = 2	x/y = lat/lon (degE7), z = altitude (m, default 50)	Vehicle flies to the point in guided mode
Velocity	SET_POSITION_TARGET_LOCAL_NED, frame LOCAL_NED, velocity mask	vx, vy, vz (m/s)	Vehicle follows velocity
Set home	COMMAND_LONG MAV_CMD_DO_SET_HOME	param1 = 0 (use given location); param5/6 = lat/lon (deg); param7 = 0	Sent at connection from the operator's configured home, and about every 5 s when a vessel position feed is configured for ship-launched RTL

- Flight modes are decoded using the standard ArduCopter, ArduPlane and PX4 tables. A custom firmware that uses different mode numbers will display and select modes incorrectly - tell us if so.
- Only one arm or disarm may be awaiting an ACK at a time (they share a command id). A command without an ACK within 5 s is retried, up to 2 attempts, then reported as failed.
- Operator permissions (role-based) and the audit log are enforced on the server; your firmware does not need to know about them.
- If your vehicle must not receive periodic DO_SET_HOME (for example a land-based operation), tell us before integration so we confirm the behaviour for your deployment.

8. Timing, supervision and failure behaviour

Item	Value	What it means for you
First heartbeat wait	6-10 s after the server opens the listener (depends on how the connection is started)	Keep sending; the server may open the listener after your first packets
Command ACK timeout	5 s, 2 attempts	Answer COMMAND_LONG promptly; reply with COMMAND_ACK, including rejections
Link-loss detection	10 s with no message	Server marks the vehicle disconnected and stops sending its heartbeat
Read-error limit	10 consecutive errors	Server drops the link
Automatic re-connect	About every 15 s	After a drop, resume sending and the server reconnects by itself; stream rates are requested again
Server heartbeat	1 Hz from system 255, only while connected	Your GCS-loss failsafe (for example ArduPilot FS_GCS_ENABLE) is fed only while the link is up

SAFETY: DroneArjuna is not a flight-safety system. The vehicle must be safe on its own when the link is lost: configure the autopilot's GCS-loss, battery, geofence and RTL failsafes for the intended operation. Mobile and satellite links will drop; verify the behaviour in the acceptance tests (Section 11).

9. Security: MAVLink 2 signing (mandatory for hosted use)

The server accepts UDP datagrams from any source address on the assigned port. **Signing is therefore the only authentication** between your vehicle and the platform. A vehicle that is not signed will be ignored, and an attacker without the key cannot inject commands.

- With signing on, the server signs every outgoing frame and discards every incoming frame that is unsigned or has a bad signature. The only exception is RADIO_STATUS.
- **The key is one 32-byte secret, given as 64 hex characters, shared by the platform and all vehicles of the deployment.** Treat it like a password: do not commit it to a repository or bake it into a firmware image; load it into the autopilot at provisioning.
- Key rotation changes the key on the platform and on every vehicle; we will schedule it with you.
- Enable signing on the vehicle **before** the first connection to the production server, and test it on the staging server first (Section 11).

Loading the key into the autopilot

Use a direct, trusted link to the autopilot (USB or serial), not the internet path. Example with pymavlink - verify the procedure and any link restrictions for your firmware version before relying on it:

```
import os, time
os.environ["MAVLINK20"] = "1"
from pymavlink import mavutil

key = bytes.fromhex(os.environ["DA_SIGNING_KEY"])        # 64 hex chars, supplied by DroneArjuna
m = mavutil.mavlink_connection("/dev/ttyACM0", baud=115200)
m.wait_heartbeat()
ts = int((time.time() - 1420070400) * 100000)            # 10 microsecond units since 2015-01-01
m.mav.setup_signing_send(m.target_system, m.target_component, list(key), ts)
```

MAVProxy's `signing setup <passphrase>` derives its own key from the passphrase and will **not** produce our key unless you use the raw key. Always confirm with the self-check tool in Appendix B that the vehicle and the DroneArjuna key match.

10. Integration procedure

#	Step	Owner
1	Exchange the information in Section 3 and agree the vehicle SYSID, port and key delivery	Both
2	Configure the autopilot: MAVLink 2, SYSID, stream rates from Section 6, failsafes (Appendix A)	Vehicle vendor
3	Run the self-check tool (Appendix B) on the bench, unsigned, until it reports PASS	Vehicle vendor
4	Load the signing key; run the self-check again with --key until PASS	Vehicle vendor
5	Configure the companion computer / modem to send only the autopilot's MAVLink to server:port (Appendix C)	Vehicle vendor
6	Register the vehicle in DroneArjuna (call sign, type, serial number, MAVLink ID = SYSID)	DroneArjuna
7	Connect to the staging server; confirm live telemetry in the web UI	Both
8	Run the acceptance tests in Section 11 on the ground	Both
9	Move to the production server details; repeat tests 1-3 of Section 11	Both
10	First flight under the agreed operating procedure	Vehicle vendor / operator

11. Acceptance tests (ground, propellers removed)

#	Test	Pass criterion
1	Connect	Vehicle appears connected in the web UI within 30 s of the operator pressing Connect (or automatically)
2	Telemetry	Position, attitude, speed, battery and GPS update at the Section 6 rates; units correct
3	Flight mode	Mode shown matches the autopilot; changing mode from the UI changes it on the vehicle
4	Arm / disarm	COMMAND_ACK received; armed state in the UI matches the vehicle; arm refused with no GPS fix
5	Identity	Only one heartbeat source on the link; the vehicle shown is the correct SYSID
6	Signing	With the key: normal operation. With a wrong or missing key: no data is accepted
7	Link loss	Disconnect the modem for 15 s: server shows the link lost, vehicle executes its configured failsafe
8	Link recovery	Restore the link: the vehicle reconnects automatically within about 30 s and telemetry resumes
9	Vehicle reboot	Reboot the autopilot: telemetry returns at full rates without operator action
10	Command latency	Round-trip of an arm command measured and recorded for the real link

12. Troubleshooting

Symptom	Likely cause	Action
Server never shows the vehicle	Wrong host/port, firewall or carrier blocking UDP, vehicle not sending	Capture packets on the vehicle router; verify outbound UDP; try the staging server
Connects, then drops after about 10 s	One-way link (vehicle to server works, return path blocked), NAT mapping expired, long gaps	Check the vehicle receives the server's 1 Hz heartbeat; keep traffic continuous
Heartbeat but no position / battery	Streams not configured or not enabled after boot	Set message rates in firmware (Section 6) instead of relying on the request
No data after enabling signing	Key mismatch, key not loaded, or MAVLink 1 frames	Re-load the key; run the self-check with --key; confirm MAVLink 2
Flight mode flickers or is wrong	More than one component sends HEARTBEAT	Filter other components at the companion computer (Section 5.3)
Mode shows as MODE_n	Autopilot type not ArduPilot/PX4	Tell us - mode table must be added
Arm denied by the server	Battery under 20%, or no 3D fix / fewer than 6 satellites	Fix GPS / battery reporting
Commands time out	Vehicle does not reply with COMMAND_ACK, or return path blocked	Check the autopilot answers COMMAND_LONG; check NAT keep-alive

Appendix A. ArduPilot parameter examples

NOTE: Parameter names below are general ArduPilot knowledge, not taken from the DroneArjuna code. Confirm them for your firmware version. PX4 uses different parameters (for example MAV_SYS_ID and the MAVLink instance settings).

Parameter	Purpose
SYSID_THISMAV	Unique vehicle system id (must equal the id in Section 3)
SYSID_MYGCS	255 (the server's id)
SERIALn_PROTOCOL = 2, SERIALn_BAUD	MAVLink 2 on the port toward the companion computer or modem
SRn_POSITION, SRn_EXTRA1, SRn_EXTRA2, SRn_EXT_STAT, SRn_RAW_SENS, SRn_RC_CHAN	Default stream rates on that port; set to match Section 6 (ATTITUDE 10 Hz, position 4 Hz, and so on)

Parameter	Purpose
FS_GCS_ENABLE	Failsafe on loss of GCS heartbeat; choose per operation
FS_BATT_ENABLE, FENCE_ENABLE, RTL_ALT	Battery, geofence and return behaviour on the vehicle itself

Appendix B. Self-check tool (da_link_check.py)

A small Python tool that behaves like the DroneArjuna server toward your vehicle: it listens on a UDP port as system id 255 in MAVLink 2, sends the 1 Hz heartbeat, requests the same stream rates, observes your vehicle for 20 s and prints PASS or FAIL. It never sends arm, mode or takeoff commands. Run it on any machine your vehicle can reach; point the vehicle's UDP output at that machine.

```
pip install pymavlink
python da_link_check.py --port 14550                      # unsigned bench test
python da_link_check.py --port 14550 --key <64 hex chars>  # with signing
```

It checks: first heartbeat arrives; each required message in Section 6 meets at least 80% of its rate; only one component sends HEARTBEAT; autopilot type is ArduPilot or PX4; and, with a key, that signed frames are accepted. Exit code 0 means pass.

Appendix C. Forwarding from a companion computer

Typical case: the autopilot is wired to a companion computer with a cellular modem. Example mavlink-router configuration that sends the autopilot's MAVLink out to the server as a client (verify option names for your version):

```
# /etc/mavlink-router/main.conf
[UartEndpoint autopilot]
Device = /dev/ttyACM0
Baud   = 921600

[UdpEndpoint dronearjuna]
Mode    = Normal                  # client: sends to Address:Port
Address = <server hostname or IP>
Port    = <assigned port>
```

- Make sure no other MAVLink component (camera, gimbal, companion application) injects its own HEARTBEAT into this endpoint.
- Do not run another ground station against the same autopilot on the same flow with system id 255; command ACKs and the GCS heartbeat will collide.
- Start the forwarder at boot and let it retry forever; link drops are expected on mobile networks.

For questions or to schedule the staging test, contact the DroneArjuna integration contact listed in Section 3.